

Step 2 — Style Reviewer With RAG — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Nudges, not answers — try genuinely hard before Hint 2. Each hint has 3 depth levels: **Basic**, **Intermediate**, **Advanced**.

- [Hint 1 — Why RAG, specifically, for this reviewer](#)
- [Hint 2 — A nudge on testing, not the code](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — Why RAG, specifically, for this reviewer

Basic Version

Doc08's RAG pattern exists for exactly the situation this reviewer is in: the model needs to check something against *your* real document, not whatever it already believes from training. Go back to Doc08's Core Concepts before writing anything — the pipeline (load a document, chunk it, embed it, retrieve relevant chunks, ground the answer in them) is the same one you'd use here, pointed at your style guide instead of whatever Doc08's own example used it on.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Write (or find) an actual 1-2 page style guide first — specific enough that a diff can genuinely violate it (naming conventions, docstring requirements, import ordering, line length, whatever you actually care about). A vague style guide can't be violated in a way a reviewer could ever catch, which would make this whole step untestable no matter how correctly you build the RAG plumbing around it.

Think about what "grounded" actually needs to mean for your test to prove something: if you ask the Style reviewer about a rule your style guide doesn't even mention, what should happen? That's a real test case worth designing on purpose, not an edge case to stumble into by accident.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

The README's problems table names the exact failure this step exists to prevent: "the Style reviewer disagrees with your actual style guide, because it's answering from general training knowledge instead of your specific guide." That failure is *silent* — a model confidently citing a rule that sounds plausible but isn't in your document looks identical, from the outside, to one correctly grounded in real retrieved text. How would you actually catch that in a test, rather than just hoping your prompt's wording prevents it? Think about what you'd need to log or inspect to tell the difference between "answered from the retrieved chunks" and "answered from general knowledge that happened to agree."

Also worth thinking through: Step 2's title says "that reviewer, turned into an agent" — Security, specifically, not Style. What would actually make Security *agent*-shaped now, when Step 1 built it as a plain chain? What's different about Security's job that a tool would help with, that RAG-grounding wouldn't?

Difference between Basic, Intermediate, and Advanced: Basic points you back at the doc that already teaches this whole pattern. Intermediate asks you to build a style guide specific enough to actually be testable, and to design a test for the "ungrounded" case on purpose. Advanced asks you to think about *proving* groundedness, not just hoping for it, and separately, to reconsider what Step 1's Security reviewer is actually missing that would make it agent-shaped.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — A nudge on testing, not the code

Basic Version

Test both reviewers — Security and Style — separately, against the *same* small set of example diffs, before either one ever needs to work alongside the other. That's the same "prove it alone first" discipline every earlier project used, just without this document walking you through each sub-step of doing it.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

For a genuinely convincing test of "grounded, not guessing," you need a rule your style guide states in a way that's *unusual* enough that a model wouldn't invent it from general training knowledge on its own (a generic-sounding rule like "use descriptive variable names" doesn't prove much either way — most models would say that anyway, guide or no guide). Write one deliberately specific/unusual rule into your style guide, plant a diff that violates only that rule, and confirm the reviewer catches it. That's a much stronger proof of real grounding than a rule the model would've gotten right regardless.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Try logging (or printing, for now) exactly which chunks your retriever returns for a given diff, before the model ever sees them, and read them yourself. If the retrieved chunks genuinely don't contain anything relevant to what the model ends up saying, that's a real, catchable sign of the model falling back on general knowledge instead of what was actually retrieved — worth building this visibility in now, not after you've already shipped a Style reviewer you can't actually verify is grounded.

Difference between Basic, Intermediate, and Advanced: Basic says to test the two reviewers apart, the same as every earlier project. Intermediate pushes you to design a specific test that could only pass if grounding is real. Advanced pushes you to build the visibility (inspecting retrieved chunks directly) that lets you verify groundedness yourself, rather than trusting your prompt's wording or a test that could still pass by coincidence.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)